

Module 2 — Analyse et Conception

Sécurité par Conception & Architecture Performante

Semaine 9 · Cours 2 · GL-EN3-2026

Lundi 25 Mai 2026

Plan du cours

Semaine 9 · Cours 2

Objectif : connaître les termes, comprendre les risques, savoir quoi chercher — et compléter Section 11 du Projet 1

1 Sécurité par Conception

OWASP Top 10:2025 — les 10 vulnérabilités avec mitigations + Security by Design

3 Architecture Performante

Caching · Indexation BDD · Scalabilité horizontale · Load Balancing

2 AuthN vs AuthO

JWT · OAuth2 · RBAC vs ABAC — authentication et autorisation en pratique

4 Accessibilité & ATAM

WCAG 2.1 P.O.U.R. · Revue ATAM simplifiée · ADR · Section 11 Checklist Projet 1

Partie 1

Sécurité par Conception

OWASP Top 10:2025 · Security by Design · AuthN/AuthO

Security by Design — 3 Principes Fondateurs

La sécurité ne s'ajoute pas après coup — elle se conçoit dès l'architecture

LP Least Privilege

Définition

Chaque composant n'accède qu'à ce dont il a strictement besoin.

Exemple

Un service de lecture n'a PAS le droit d'écrire. Un user ne voit PAS les données des autres.

DD Defense in Depth

Définition

Plusieurs couches de protection indépendantes — si une tombe, les autres tiennent.

Exemple

Validation côté client + côté serveur + contraintes BDD + pare-feu applicatif.

FS Fail Secure

Définition

En cas d'erreur, le système refuse l'accès — jamais l'autoriser par défaut.

Exemple

Une exception dans le contrôle d'accès → 403 Forbidden. Jamais un accès fantôme.

Threat Modeling (STRIDE) : avant de coder, identifier les acteurs malveillants, les surfaces d'attaque, et les contrôles nécessaires.
Spoofing, Tampering, Repudiation, Information disclosure, Denial of service, Elevation of privilege

OWASP Top 10 : 2025 — Vue d'ensemble

Référence mondiale des 10 vulnérabilités web les plus critiques — mise à jour 2025

Rang	Vulnérabilité	Nouveauté vs 2021	Niveau
A01	Broken Access Control	Maintenu #1 — 100% des apps	Critique
A02	Security Misconfiguration	Monte de #5 -> #2	Critique
A03	Software Supply Chain Failures	NOUVEAU 2025	Haute
A04	Cryptographic Failures	Descend de #2 -> #4	Haute
A05	Injection	Descend de #3 -> #5	Haute
A06	Insecure Design	Descend de #4 -> #6	Haute
A07	Authentication Failures	Maintenu #7	Haute
A08	Software or Data Integrity Failures	Maintenu #8	Moyenne
A09	Security Logging & Alerting Failures	+ "Alerting" au nom	Moyenne
A10	Mishandling of Exceptional Conditions	NOUVEAU 2025	Moyenne

A01:2025

Broken Access Control

OWASP Top 10:2025

Risques & Symptômes

- Bypass via manipulation d'URL : /users/123 -> /users/456 (IDOR)
- API sans contrôle sur POST/PUT/DELETE — endpoints accessibles sans droits
- Elévation de privilège : accès admin sans être admin
- CORS (Cross-Origine Resource Sharing) mal configuré — origines non autorisées acceptées

Comment prévenir

- Refuser par défaut (deny by default) — accorder l'accès explicitement
- Vérification serveur : ne jamais faire confiance au client pour ses propres droits
- RBAC strict : vérifier le rôle à chaque endpoint, pas seulement à la connexion. (Role-Based Access Control)
- JWT à durée de vie courte + invalider les sessions à la déconnexion
- Logger les accès refusés et alerter sur les tentatives répétées

Projet 1 : Vérifier dans CHAQUE endpoint que l'utilisateur connecté peut accéder à la ressource demandée

A02:2025

Security Misconfiguration

OWASP Top 10:2025

Risques & Symptômes

- Monte de #5 à #2 — 100% des apps testées, taux moyen 3%
- Comptes et mots de passe par défaut non changés (admin/admin)
- Messages d'erreur révèlent la stack technique (stack trace en production)
- Fonctionnalités inutiles activées : ports, services, pages de debug
- Secrets (clés API) dans le code source ou les fichiers de config versionnés

Comment prévenir

- Configuration 'secure by default' — désactiver tout ce qui n'est pas nécessaire
- Variables d'environnement pour les secrets — jamais dans le code source
- En-têtes HTTP de sécurité : Content-Security-Policy, HSTS, X-Frame-Options
- Infrastructure as Code : dev/QA/prod configurés identiquement
- Supprimer la doc et les exemples de démo en production

Projet 1 : Utiliser .env pour les secrets, ajouter .env au .gitignore, configurer les headers HTTP dès le départ

Software Supply Chain Failures

OWASP Top 10:2025

Risques & Symptômes

- NOUVEAU 2025 — taux d'incidence le plus élevé (5.19%), voté #1 en communauté
- Dépendances avec des CVEs connus non mises à jour (Log4Shell, CVE-2021-44228)
- Pipeline CI/CD compromis — 1 dev peut pousser en prod sans revue (SolarWinds 2019)
- Packages téléchargés depuis des sources non officielles ou non signés
- Bybit 2025 : 1,5 milliard \$ volés via une attaque supply chain sur un wallet

Comment prévenir

- Inventaire des dépendances : SBOM (Software Bill of Materials)
- npm audit / pip check / OWASP Dependency-Check régulièrement
- Surveiller CVE : nvd.nist.gov et osv.dev — abonnement aux alertes de sécurité
- Séparation des responsabilités dans CI/CD — revue obligatoire avant merge
- Déploiements progressifs (canary) pour limiter l'exposition

Projet 1 : Lancer npm audit ou pip check régulièrement. Documenter vos dépendances et versions dans §11

Risques & Symptômes

- Données sensibles transmises en HTTP — pas de chiffrement en transit
- Mots de passe stockés en MD5 ou SHA1 (cassables en secondes avec GPU)
- Clés API ou secrets hardcodés dans le code source ou GitHub
- Certificats TLS non validés, algorithmes obsolètes (DES, RC4)
- Données sensibles en cache sans protection (cookies, logs)

Comment prévenir

- HTTPS partout — TLS 1.2 minimum, TLS 1.3 recommandé
- Mots de passe hashés avec bcrypt, Argon2 ou PBKDF2 — jamais MD5/SHA1
- Chiffrement des données sensibles au repos (AES-256)
- Secrets dans un gestionnaire (HashiCorp Vault, .env hors du repo Git)
- Désactiver le cache HTTP pour les réponses contenant des données sensibles

Projet 1 : Stocker les mots de passe hashés (bcrypt), utiliser HTTPS, mettre les secrets dans .env hors du repo

A05:2025

Injection (SQL · XSS · Command)

OWASP Top 10:2025

Risques & Symptômes

- Entrée utilisateur non validée interprétée comme une commande
- SQL Injection : `SELECT * FROM users WHERE email = " OR '1'='1'`
- XSS : afficher du contenu utilisateur sans échapper crée des scripts injectés
- Command Injection : `exec('nslookup '+userInput) → exec('nslookup x; cat /etc/passwd')`
- 37 CWEs mappés, >62 000 CVEs — catégorie la plus testée (100% des apps)

Comment prévenir

- Requêtes paramétrées (prepared statements) — JAMAIS de concaténation SQL
- Utiliser un ORM (Sequelize, Prisma, Hibernate) qui paramétrise automatiquement
- Validation et sanitisation des entrées côté SERVEUR (jamais seulement côté client)
- Échappement des sorties dans le HTML — Content-Security-Policy (CSP)
- Principe de moindre privilège sur les comptes de base de données

Projet 1 : Ne jamais concaténer des variables dans des requêtes SQL — utiliser l'ORM ou les prepared statements

Risques & Symptômes

- Failles dans l'architecture, pas le code — même du code parfait reste vulnérable
- Logique métier non sécurisée : booking de 600 sièges en 1 requête (attaque tarifaire)
- Récupération de mot de passe par questions secrètes (interdit par NIST 800-63b)
- Pas de Threat Modeling — les attaquants ont plus réfléchi au système que les développeurs
- Fonctionnalités sans protection contre les bots ou le brute force

Comment prévenir

- Threat Modeling dès la conception — framework STRIDE
- Intégrer la sécurité dans les User Stories (cas d'abus, misuse cases)
- Revue d'architecture avec critères de sécurité explicites
- Rate limiting et quotas planifiés dès la conception
- Tests d'abus en plus des cas nominaux — pas seulement le happy path

Projet 1 : Pendant la revue ATAM : poser 'Que se passe-t-il si un attaquant abuse de ce flux ?'

A07:2025

Authentication Failures

OWASP Top 10:2025

Risques & Symptômes

- Credential stuffing : listes de mots de passe compromis (Have I Been Pwned)
- Brute force non bloqué — pas de rate limiting sur /login
- Sessions non invalidées à la déconnexion côté serveur
- Session ID dans l'URL (visible dans les logs, Referer header)
- Absence de MFA pour les comptes sensibles ou administrateurs

Comment prévenir

- MFA (Multi-Factor Authentication) pour les comptes à privilèges
- Rate limiting : bloquer après 5 tentatives, délai progressif
- Sessions invalidées côté serveur à la déconnexion — pas seulement cookie supprimé
- Session IDs dans cookies HttpOnly + Secure — jamais dans l'URL
- Utiliser une lib éprouvée (Passport.js, Spring Security) — ne pas réimplémenter l'auth

Projet 1 : Utiliser Passport.js ou l'équivalent. Ne JAMAIS réimplémenter l'authentification de zéro

A08:2025

Software or Data Integrity Failures

OWASP Top 10:2025

Risques & Symptômes

- Mises à jour téléchargées et appliquées sans vérification de signature
- Désérialisation de données non fiables (Java serialization, Python pickle)
- Bibliothèques CDN sans vérification d'intégrité (SRI hash manquant)
- CI/CD sans contrôle d'intégrité des artefacts de build
- Cookies ou tokens sérialisés, modifiables par l'utilisateur sans vérification

Comment prévenir

- Signatures numériques sur les packages et les mises à jour applicatives
- Ne jamais désérialiser des données non fiables sans validation préalable
- Préférer JSON avec validation de schéma à la sérialisation native
- Subresource Integrity (SRI) sur les scripts CDN : integrity='sha384-...'
- Revue de code obligatoire avant merge vers les branches de production

Projet 1 : JWT signé avec un secret fort côté serveur. Ne pas stocker d'état côté client sans signature

Risques & Symptômes

- Sans logs, les attaques ne sont pas détectées — certaines brèches passent inaperçues des années
- Événements critiques non loggés : tentatives de login échouées, accès refusés
- Logs incomplets — impossible de reconstituer une attaque a posteriori
- DANGER : logs qui contiennent des mots de passe, tokens ou données PII
- Pas d'alertes configurées sur les comportements anormaux

Comment prévenir

- Logger : authentications (succès ET échecs), changements de rôle, accès sensibles
- Logs structurés JSON avec : timestamp, userId, action, ressource, résultat, IP
- Centralisation des logs (ELK Stack, Datadog) — jamais uniquement en local
- Alertes sur les patterns suspects : trop de tentatives, heures inhabituelles
- Ne JAMAIS logger de données sensibles (mots de passe, tokens, cartes bancaires)

Projet 1 : Implémenter un middleware de logging qui enregistre chaque requête HTTP avec code de réponse + userId

Mishandling of Exceptional Conditions

OWASP Top 10:2025

Risques & Symptômes

- NOUVEAU 2025 — 24 CWEs, programmes qui ne gèrent pas les situations anormales
- Failing Open : une exception dans le contrôle d'accès accorde l'accès — CRITIQUE
- Stack trace exposée à l'utilisateur — révèle l'architecture interne au attaquant
- Transaction partielle non rollbackée — incohérence financière ou corruption de données
- Ressources non libérées après exception — épuisement mémoire (DoS progressif)

Comment prévenir

- FAIL CLOSED : en cas d'erreur, refuser l'accès — jamais l'autoriser par défaut
- Gestionnaire d'exceptions global en dernier recours (catch-all)
- Messages d'erreur génériques côté client, logs détaillés côté serveur uniquement
- Rollback systématique des transactions en cas d'erreur partielle
- Rate limiting et quotas pour prévenir l'épuisement des ressources

Projet 1 : Chaque endpoint : try/catch retournant 400/403/500 sans exposer les détails internes

OWASP Top 10:2025 — Récapitulatif des mitigations

Une règle par vulnérabilité — à appliquer dès Projet 1

Code	Vulnérabilité	Mitigation principale
A01	Broken Access Control	Deny by default · RBAC · vérification serveur
A02	Misconfiguration	.env pour secrets · headers HTTP · désactiver debug
A03	Supply Chain	npm audit · SBOM · revue CI/CD obligatoire
A04	Cryptographic Failures	bcrypt · HTTPS · AES-256 · pas de secrets dans Git
A05	Injection	Requêtes paramétrées · ORM · validation serveur
A06	Insecure Design	Threat Modeling · misuse cases · rate limiting
A07	Auth Failures	MFA · rate limiting login · lib d'auth éprouvée
A08	Integrity Failures	Signatures numériques · SRI · revue de code
A09	Logging Failures	Logs structurés · centralisation · alertes
A10	Exceptional Conditions	Fail closed · rollback · messages génériques

Partie 1 — Suite

AuthN vs AuthO

JWT · OAuth2 · RBAC · ABAC

Authentication (AuthN) vs Autorisation (AuthO)

Deux concepts distincts — souvent confondus, toujours différents

Authentication (AuthN) — "Qui es-tu ?"

Vérifie l'IDENTITÉ de l'utilisateur.

- Login/mot de passe, biométrie, certificat
- L'utilisateur prouve qu'il est bien qui il prétend être
- Résultat : un token ou une session identifiant l'utilisateur
- Exemples : bcrypt + JWT, OAuth2, SAML, LDAP

Autorisation (AuthO) — "Qu'as-tu le droit de faire ?"

Vérifie les PERMISSIONS d'une identité connue.

- Se produit APRÈS l'authentification — l'identité est déjà connue
- L'utilisateur connu peut-il effectuer CETTE action sur CETTE ressource ?
- Résultat : accès accordé (200) ou refusé (403 Forbidden)
- Exemples : RBAC, ABAC, ACL, OAuth2 scopes

Analogie : Badge employé = AuthN (prouve qui vous êtes). Accès aux salles (RH, Serveur, Direction) = AuthO (votre rôle détermine vos droits).

JWT — JSON Web Token

Standard de facto pour les APIs stateless

Structure (Header.Payload.Signature)

Header

Algorithme : HS256 (symétrique) ou RS256 (asymétrique)

Payload

Claims : userId, role, exp, iss, aud — base64 lisible par tous !

Signature

HMAC(header+payload, secret) — garantit l'intégrité

Stateless : le serveur valide la signature sans consulter la BDD

Bonnes pratiques

- Durée de vie courte : 15–60 min + refresh tokens
- Secret fort : 256 bits minimum — jamais 'mysecret'
- Valider exp, iss, aud à chaque requête
- Ne PAS stocker de données sensibles dans le payload
- Algorithme : RS256 préféré pour les microservices
- Authorization: Bearer <token> dans l'en-tête HTTP

OAuth2 & Modèles d'Autorisation

Délégation · RBAC · ABAC

OAuth2 — Délégation d'autorisation

Resource Owner	L'utilisateur qui possède les données
Client	Votre application qui veut y accéder
Auth Server	Google, GitHub, Auth0... délivre les tokens
Resource Server	L'API protégée qui vérifie les tokens
Exemple : "Se connecter avec Google" — Google AuthN, votre app reçoit un access_token	

RBAC vs ABAC

	RBAC	ABAC
Logique	Permissions liées aux rôles	Permissions liées aux attributs
Complexité	Faible — simple à implémenter	Élevée — très flexible
Exemple	Roles: Étudiant, Enseignant, Admin	Accès si propriétaire ET heure < 18h
Usage	Applications standard — Projet 1 ici	Systèmes complexes, gouvernement

Projet 1 : RBAC est suffisant. Implémenter tables users + roles + user_roles. OAuth2 si connexion via Google/GitHub.

Partie 2

Architecture Performante

Caching · Indexation · Scalabilité · Load Balancing

Architecture pour la Performance — 3 Leviers

La performance se conçoit dès l'architecture — pas en post-optimisation

1 Caching

Ne pas recalculer ou recharger ce qui ne change pas souvent.

- Réduire les appels à la BDD
- Stocker en mémoire (Redis)
- Cache HTTP pour les réponses GET

Cacher ce qui est lu souvent et change rarement.

2 Indexation BDD

Accélérer les recherches sur les colonnes fréquemment filtrées.

- Sans index : Full Table Scan $O(n)$
- Avec index B-Tree : $O(\log n)$
- Index sur WHERE, JOIN, ORDER BY

Indexer les colonnes de filtrage — pas toutes les colonnes.

3 Scalabilité

Adapter la capacité du système à la charge.

- Scale Up : plus de RAM/CPU (limité)
- Scale Out : plus d'instances (cloud-native)
- Stateless requis pour Scale Out

Rendre l'app stateless en amont — sessions dans Redis.

Règle 80/20 : 80% des problèmes de performance viennent de 20% du code. Mesurer avant d'optimiser.

Caching — Stratégies et Niveaux

Cache-Aside · Redis · HTTP Cache · CDN

Niveau	Outil	Ce qu'on cache	Invalidation
Applicatif	Redis, Memcached	Résultats BDD, sessions, calculs lourds	TTL ou événement
HTTP	Cache-Control, ETags	Réponses GET statiques ou semi-statiques	max-age, ETag
BDD	Query cache, ORM	Requêtes SQL répétitives identiques	Automatique
CDN	Cloudflare, Fastly	Assets statiques (JS, CSS, images)	TTL ou purge manuelle

Pattern Cache-Aside (le plus courant)		
1 Requête arrivée	2 Cache HIT	3 Cache MISS
L'app vérifie d'abord Redis (cache hit?)	Données retournées depuis Redis — très rapide	Aller en BDD, stocker en cache, retourner

Indexation de Base de Données

Accélérer les recherches — Full Table Scan $O(n)$ vs B-Tree $O(\log n)$

Quand INDEXER — $O(\log n)$

- Colonne dans les clauses WHERE fréquentes
- Colonne utilisée dans les JOIN (clés étrangères)
- Colonne utilisée dans ORDER BY
- Colonne de recherche de texte (full-text index)

Quand NE PAS indexer

- Colonne rarement utilisée en filtre
- Tables très petites (scan complet est plus rapide)
- Colonne booléennes (peu de valeurs distinctes)
- Tables à très forte charge d'écriture (index ralentit INSERT/UPDATE)

Types d'index courants

B-Tree

Défaut — égalité et range ($=$, $<$, $>$, BETWEEN)

Unique

Garantit l'unicité (email, username)

Composite

Plusieurs colonnes (user_id, created_at)

Full-text

Recherche dans du texte libre

Problème N+1 à éviter

1 requête pour 100 cours + 100 requêtes pour les inscrits = 101 requêtes ! Solution : JOIN ou eager loading.

Scalabilité — Scale Up vs Scale Out

Statelessness · Load Balancing · Connection Pooling

Critère	Scale Up (Vertical)	Scale Out (Horizontal)
Approche	Augmenter la puissance d'1 machine	Ajouter des instances identiques
Coût	Élevé et limité physiquement	Flexible — payer à l'usage (cloud)
Résilience	Single point of failure	Résilient — si 1 instance tombe, les autres continuent
Prérequis	Aucun	Application STATELESS obligatoire

Load Balancer (Nginx, HAProxy, AWS ELB) : distribue les requêtes entre instances

Algo: Round Robin (séquentiel) · Least Connections (moins chargé) · IP Hash (même client → même serveur)

Stateless requis : sessions → Redis, fichiers uploadés → S3/MinIO, config → variables d'environnement



Partie 3

Accessibilité & ATAM

WCAG 2.1 · ATAM simplifié · ADR · §11 Projet 1

Accessibilité Web — WCAG 2.1 : Les 4 Principes P.O.U.R.

Web Content Accessibility Guidelines — Standard international — Cible : niveau AA

P Perceptible	O Opérable	U Understandable	R Robust
Alt text sur toutes les images informatives	Navigation clavier complète (Tab, Enter, Espace)	Langue déclarée : <code><html lang='fr'></code>	HTML sémantique : header, nav, main, footer
Contraste 4.5:1 minimum (texte normal)	Focus visible — ne pas faire <code>outline:none</code>	Labels explicites sur tous les champs	ARIA roles quand nécessaire (<code>role='alert'</code>)
Sous-titres sur les vidéos	Pas de limite de temps stricte	Messages d'erreur indiquant comment corriger	Ne pas réimplémenter les composants natifs
Ne pas transmettre l'info par la couleur seule	Liens avec texte descriptif (pas 'cliquez ici')	Navigation prévisible et cohérente	Tester avec lecteurs d'écran (NVDA, VoiceOver)

Projet 1 : Cible WCAG 2.1 niveau AA. Contraste, navigation clavier, labels formulaires, HTML sémantique — documenter dans Section 11.

ATAM Simplifié & ADR

Architecture Tradeoff Analysis Method · Architectural Decision Record

ATAM — 5 Etapes adaptées au Projet 1

1

Présenter l'architecture

Motivation Business, Diagramme de composants, flux principaux

2

Lister les attributs qualité

Performance, sécurité, maintenabilité, accessibilité

3

Identifier les décisions clés

API style, ORM, auth, BDD, hébergement

4

Analyser les trade-offs

Pour chaque décision : avantages et inconvénients

5

Documenter en ADR

1 Architectural Decision Record par décision majeure

Format ADR (Architectural Decision Record)

ADR-001 : [Titre]

Statut : *Proposé* / *Accepté* / *Déprécié*

Contexte

Pourquoi cette décision est nécessaire.

Options considérées

- Option A : avantages / inconvénients

- Option B : avantages / inconvénients

Décision

Ce qui a été choisi et POURQUOI.

Conséquences

+ Impact positif / - Risque à surveiller

Projet 1 : Rédiger au moins 1 ADR — ex : ADR-001 choix monolithe modulaire vs microservices

Section 11 — Checklist Choix Technologiques

Ce que chaque équipe doit documenter dans le Projet 1

Architecture & API	Sécurité OWASP	Performance	Accessibilité & Qualité
Pattern architectural justifié (ADR)	AuthN/AuthO : JWT + RBAC	Stratégie de caching (1 niveau min.)	Cible WCAG 2.1 déclarée (AA)
Diagramme Sec 10 complété	A01: vérif. serveur sur chaque endpoint	Colonnes à indexer identifiées	Contraste vérifié (4.5:1)
Style API choisi (REST recommandé)	A04: bcrypt + HTTPS + .env	Pagination sur toutes les listes	Stack technique documentée
Ébauche OpenAPI / Swagger	A05: ORM ou requêtes paramétrées	Connection pooling configuré	Dépendances listées avec justification

Objectif : chaque équipe repart avec Section 11 rempli. Temps de travail guidé en fin de cours — posez vos questions !

Synthèse du Cours 2

OWASP

10 vulnérabilités à connaître — deny by default, secrets hors du code, valider côté serveur

AuthN

Authentification = qui es-tu ? · Autorisation = qu'as-tu le droit ? — deux couches distinctes

JWT

Token signé stateless — durée courte, valider exp/iss/aud, jamais de données sensibles dans le payload

Perf

Cache-Aside (Redis) + index B-Tree + statelessness pour Scale Out — concevoir dès le départ

WCAG

P.O.U.R. — Perceptible, Opérable, Understandable, Robust — cible niveau AA pour Projet 1

Section 11

Checklist complétée : architecture, sécurité, performance, accessibilité, stack documentée + 1 ADR

Prochaines étapes

Section 10 Architecture + Section 11 Choix technologiques finalisés · 1 ADR rédigé par équipe

Sprint de finalisation Projet 1 — soumission dans 2 semaines